

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ НЕПРЕРЫВНОГО И ДИСТАНЦИОННОГО ОБРАЗОВАНИЯ

КАФЕДРА №3

ОЦЕНКА

ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О КОНТРОЛЬНОЙ РАБОТЕ №2
Вариант 14

по дисциплине: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

В3441
номер группы

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Задание

Для выполнения заданий (см. таблицу с вариантами) на обработку числовой матрицы написать программу на языке C++ с применением библиотеки STL.

- Для хранения данных использовать один из контейнеров (задано вариантом)
- Матрицу можно заполнять случайными числами или присваивать значения через инициализацию, если задание не связано с заполнением матрицы.
- Сформулировать и реализовать собственную задачу, связанную с использованием матрицы. Повторять задания из чужих вариантов, приведенных в этой контрольной, нельзя
- Обязательно применить не менее 3 функций из раздела `algorithm`.
- Каждое задание оформить как отдельную функцию. Функции, выполняющие вычисления, не должны содержать код для ввода данных и вывода на экран результата. Исходные данные передавать в функции через параметры.
- Программа должна содержать меню, позволяющее проверить выполнение каждой функции.

2 Вариант задания

Кулаков	14	list
---------	----	------

Вариант 14

В матрице размером $n \times m$ вещественных элементов:

- Вычислить количество элементов, равных 0;
- Найти в матрице сумму элементов, меньших Z.

3 Формулировка собственного задания на матрицы

Необходимо выполнить масштабирование матрицы на коэффициент k . То есть преобразовать матрицу таким образом, чтобы каждый элемент был умножен на k .

4 Перечисление использованных методов контейнеров с пояснением результатов их работы

Методы контейнера `std::list`

4.1 `push_back(value)`

- Добавляет элемент `value` в конец списка.
- Применяется при формировании каждой строки (добавляем новые элементы в конец строки) и при построении самой матрицы (добавляем готовую строку в конец матрицы).

4.2 Итерация через `for (auto& row : mat)` и `for (auto x : row)`

- Доступ к каждому элементу контейнера без явных индексов.
- Позволяет обойти всю матрицу (или строку) для подсчёта, суммирования или вывода на экран.

4.3 `row.begin() / row.end()` и `mat.begin() / mat.end()`

- Возвращают итераторы на начало и на «конец» (за последним элементом) списка.
- Нужны для передачи в алгоритмы (`std::count_if`, `std::accumulate`, `std::transform`)

5 Перечисление использованных функции из раздела `algorithm` с пояснением результатов их работы

5.1 `std::count_if(begin, end, pred)`

- Перебирает все элементы в диапазоне `[begin, end)`, считает, сколько из них удовлетворяют предикату `pred`.
- В коде используется предикат `[](double x){ return x == 0.0; }` — проверка «равен ли элемент нулю».

- Результат: число нулевых элементов в одной строке, суммируется по всем строкам

5.2 `std::accumulate(begin, end, init, op)`

- Начиная с первоначального значения `init`, проходит по всем элементам в диапазоне `[begin, end)` и накапливает результат через двоичный оператор `op(accumulator, element)`.
- В коде используется `init = 0.0`, а `op = [Z](double acc, double x){ return acc + (x < Z ? x : 0.0); }` — добавление к сумме `x` только если `x < Z`.
- Результат: сумма всех элементов строки, которые меньше `Z`; далее строки складываются в общую сумму.

5.3 `std::transform(begin, end, output_it, op)`

- Проходит по входному диапазону `[begin, end)`, применяет к каждому элементу оператор `op` и записывает результат по итератору `output_it` (например, в конец другого контейнера через `back_inserter`).
- В коде используется `op = [k](double x){ return x * k; }` — умножение каждого элемента матрицы на коэффициент `k`.
- Результат: построение новой матрицы из увеличенных значений.

6 Скриншоты с примерами работы

Введите размеры матрицы n m: 4

5

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 1

2.28	0.00	-3.43	0.00	-2.35
1.48	2.04	0.60	3.95	4.60
0.99	2.76	2.85	-3.27	-4.39
2.24	0.00	2.60	3.88	-0.60

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 2

Количество элементов, равных 0: 3

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 3

Введите Z: 2

Сумма элементов < 2.00 равна -10.96

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 4

Введите коэффициент масштабирования k: 10

Матрица масштабирована на 10.00

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 1

22.84	0.00	-34.28	0.00	-23.47
14.79	20.42	6.04	39.48	46.01
9.93	27.59	28.54	-32.70	-43.90
22.38	0.00	26.01	38.76	-6.05

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Введите размеры матрицы n m: 6

7

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 1

2.37	-2.81	3.57	3.68	1.08	0.00	-3.41
0.00	4.16	0.00	-2.44	-4.01	-4.98	-1.02
3.98	2.83	3.77	-1.25	-0.92	2.37	1.89
3.79	4.74	-0.15	3.26	-1.26	0.53	2.43
1.35	0.00	0.00	-2.69	2.74	2.90	-2.34
-3.81	-3.21	0.00	-3.88	0.00	1.65	4.28

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 2

Количество элементов, равных 0: 7

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 3

Введите Z: 1

Сумма элементов < 1.00 равна -37.64

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 4

Введите коэффициент масштабирования k: 5

Матрица масштабирована на 5.00

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов < Z
4. Масштабировать матрицу
0. Выход

Ваш выбор: 1

11.86	-14.03	17.87	18.38	5.40	0.00	-17.04
0.00	20.79	0.00	-12.19	-20.05	-24.88	-5.09
19.88	14.15	18.85	-6.25	-4.58	11.83	9.45
18.96	23.72	-0.75	16.29	-6.32	2.63	12.13
6.77	0.00	0.00	-13.45	13.71	14.50	-11.70
-19.03	-16.06	0.00	-19.41	0.00	8.27	21.39

Введите размеры матрицы n m: 2

3

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов $\leq Z$
4. Масштабировать матрицу
0. Выход

Ваш выбор: 1

0.00	-2.18	1.50
-0.39	0.01	2.95

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов $\leq Z$
4. Масштабировать матрицу
0. Выход

Ваш выбор: 2

Количество элементов, равных 0: 1

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов $\leq Z$
4. Масштабировать матрицу
0. Выход

Ваш выбор: 3

Введите Z: -10

Сумма элементов ≤ -10.00 равна 0.00

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов $\leq Z$
4. Масштабировать матрицу
0. Выход

Ваш выбор: 4

Введите коэффициент масштабирования k: 0

Матрица масштабирована на 0.00

Меню:

1. Показать матрицу
2. Подсчитать количество нулей
3. Сумма элементов $\leq Z$
4. Масштабировать матрицу
0. Выход

Ваш выбор: 1

0.00	-0.00	0.00
-0.00	0.00	0.00

7 Текст программы

```
#include <iostream>
#include <list>
#include <algorithm>
#include <numeric>
#include <iterator>
#include <cstdlib>
#include <ctime>
#include <iomanip>

// матрица - список списков вещественных чисел
typedef std::list<std::list<double>> Matrix;

// подсчет элементов, равных нулю
size_t countZeros(const Matrix& mat) {
    size_t total = 0;
    for (std::list<double> const& row : mat) {
        total += std::count_if(
            row.begin(), row.end(),
            [](double x) { return x == 0.0; }
        );
    }
    return total;
}

// сумма элементов, меньших Z
double sumLessThanZ(const Matrix& mat, double Z) {
    double total = 0.0;
    for (std::list<double> const& row : mat) {
        total += std::accumulate(
            row.begin(), row.end(), 0.0,
            [Z](double acc, double x) {
                return acc + (x < Z ? x : 0.0);
            }
        );
    }
    return total;
}

// собственная задача - масштабирование
// умножение всех элементов матрицы на коэффициент k
Matrix scaleMatrix(const Matrix& mat, double k) {
    Matrix result;
    for (std::list<double> const& row : mat) {
        std::list<double> newRow;
        // используем std::transform из <algorithm>
        std::transform(
            row.begin(), row.end(),
            std::back_inserter(newRow),
            [k](double x) { return x * k; }
        );
        result.push_back(newRow);
    }
    return result;
}

// вывод матрицы на экран
void printMatrix(const Matrix& mat) {
```



```

    for (const auto& row : mat) {
        for (double x : row) {
            // сторонняя библиотека для красивого вывода
            std::cout << std::fixed << std::setw(10) << std::setprecision(2) << x << "
";
        }
        std::cout << "\n";
    }
}

int main() {
    setlocale(LC_ALL, "Russian");

    std::srand(std::time(nullptr));
    int n, m;
    std::cout << "Введите размеры матрицы n m: ";
    std::cin >> n >> m;

    Matrix mat;
    for (int i = 0; i < n; ++i) {
        std::list<double> row;
        for (int j = 0; j < m; ++j) {
            double r = (std::rand() / (double)RAND_MAX) * 10.0 - 5.0;
            // часть элементов специально 0
            if (std::rand() % 10 == 0) r = 0.0;
            row.push_back(r);
        }
        mat.push_back(row);
    }

    int choice;
    do {
        std::cout << "\nМеню:\n";
        std::cout << "1. Показать матрицу\n";
        std::cout << "2. Подсчитать количество нулей\n";
        std::cout << "3. Сумма элементов < Z\n";
        std::cout << "4. Масштабировать матрицу\n";
        std::cout << "0. Выход\n";
        std::cout << "Ваш выбор: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                printMatrix(mat);
                break;
            case 2: {
                size_t cnt = countZeros(mat);
                std::cout << "Количество элементов, равных 0: "
                    << cnt << "\n";
                break;
            }
            case 3: {
                double Z;
                std::cout << "Введите Z: ";
                std::cin >> Z;
                double s = sumLessThanZ(mat, Z);
                std::cout << "Сумма элементов < " << Z
                    << " равна " << s << "\n";
                break;
            }
            case 4: {

```

```
        double k;
        std::cout << "Введите коэффициент масштабирования k: ";
        std::cin >> k;
        mat = scaleMatrix(mat, k);
        std::cout << "Матрица масштабирована на " << k << "\n";
        break;
    }
    case 0:
        std::cout << "Выход...\n";
        break;
    default:
        std::cout << "Неверный выбор, попробуйте снова.\n";
    }
} while (choice != 0);

return 0;
}
```